



**Universidad  
Interamericana de Panamá**  
LAUREATE INTERNATIONAL UNIVERSITIES®

Nombre: Angie Villarreal

ID: 8-1030-720

Materia: Programacion IV

Profesor: Abdel Martinez

## PROYECTO: MI BIBLIOTECA

*Aplicación de línea de comandos en Python con SQLite*

### 1. Objetivo

Desarrollar una aplicación de línea de comandos en Python para administrar una biblioteca personal, almacenando los datos en una base de datos SQLite.

### 2. Contexto

La aplicación permite registrar y administrar información sobre libros, incluyendo título, autor, género y estado de lectura. La finalidad es practicar la interacción entre Python y SQLite mediante operaciones básicas de una base de datos.

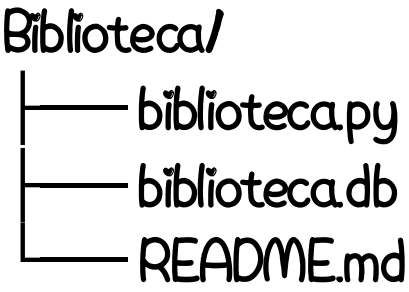
### 3. Requisitos funcionales

- Agregar nuevo libro: añadir un libro con título, autor, género y estado de lectura.
- Actualizar información de un libro: modificar cualquier campo, incluyendo el estado.
- Eliminar libro existente.
- Ver listado de libros registrados.
- Buscar libros por título, autor o género.
- Salir del programa correctamente.

### 4. Consideraciones técnicas

- Utilizar SQLite mediante la biblioteca sqlite3 de Python.
- Mantener el programa modularizado mediante funciones.
- Implementar un menú interactivo en la consola.
- Incluir comentarios explicativos.
- Incluir un archivo README.md con descripción e instrucciones de ejecución.

### 5. Estructura del proyecto



El archivo biblioteca.db se crea automáticamente al ejecutar el programa por primera vez.

### 6. Estructura de la base de datos

| Campo  | Tipo    | Descripción                    |
|--------|---------|--------------------------------|
| id     | INTEGER | Identificador único del libro. |
| título | TEXT    | Título del libro.              |
| autor  | TEXT    | Autor del libro.               |
| genero | TEXT    | Género literario.              |
| estado | TEXT    | Leído o No leído.              |

## 7. Código principal: biblioteca.py

```
import sqlite3

conexion = sqlite3.connect("biblioteca.db")
cursor = conexion.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS libros (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    título TEXT NOT NULL,
    autor TEXT NOT NULL,
    genero TEXT NOT NULL,
    estado TEXT NOT NULL CHECK(estado IN ('Leído', 'No leído'))
)
""")

conexion.commit()

def agregar_libro():
    print("\n--- AGREGAR NUEVO LIBRO ---")

    título = input("Título: ")
    autor = input("Autor: ")
    genero = input("Género: ")

    print("1. Leído")
    print("2. No leído")
    opción = input("Estado de lectura: ")

    if opción == "1":
        estado = "Leído"
    elif opción == "2":
        estado = "No leído"
    else:
        print("Opción no válida.")
        return

    cursor.execute("""
INSERT INTO libros (título, autor, genero, estado)
VALUES (?, ?, ?, ?)
""", (título, autor, genero, estado))

    conexion.commit()
    print("¡Libro agregado correctamente!")

def mostrar_libros():
    print("\n--- LISTADO DE LIBROS ---")

    cursor.execute("SELECT * FROM libros")
    libros = cursor.fetchall()

    if not libros:
        print("No hay libros registrados.")
        return

    for libro in libros:
        print("-----")
        print("ID:", libro[0])
        print("Título:", libro[1])
```

```
print("Autor:", libro[2])
print("Género:", libro[3])
print("Estado:", libro[4])
```

```
def actualizar_libro():
```

```
    print("\n--- ACTUALIZAR LIBRO ---")
```

```
    try:
```

```
        id_libro = int(input("Ingrese el ID del libro: "))
```

```
    except ValueError:
```

```
        print("Debe ingresar un número.")
```

```
    return
```

```
    cursor.execute("SELECT * FROM libros WHERE id = ?", (id_libro,))
```

```
    libro = cursor.fetchone()
```

```
    if libro is None:
```

```
        print("No se encontró el libro.")
```

```
    return
```

```
    print("\n¿Qué desea modificar?")
```

```
    print("1. Título")
```

```
    print("2. Autor")
```

```
    print("3. Género")
```

```
    print("4. Estado de lectura")
```

```
    opcion = input("Seleccione una opción: ")
```

```
    if opcion == "1":
```

```
        nuevo_valor = input("Nuevo título: ")
```

```
        cursor.execute("UPDATE libros SET título = ? WHERE id = ?", (nuevo_valor, id_libro))
```

```
    elif opcion == "2":
```

```
        nuevo_valor = input("Nuevo autor: ")
```

```
        cursor.execute("UPDATE libros SET autor = ? WHERE id = ?", (nuevo_valor, id_libro))
```

```
    elif opcion == "3":
```

```
        nuevo_valor = input("Nuevo género: ")
```

```
        cursor.execute("UPDATE libros SET genero = ? WHERE id = ?", (nuevo_valor, id_libro))
```

```
    elif opcion == "4":
```

```
        print("1. Leído")
```

```
        print("2. No leído")
```

```
        estado = input("Nuevo estado: ")
```

```
    if estado == "1":
```

```
        nuevo_valor = "Leído"
```

```
    elif estado == "2":
```

```
        nuevo_valor = "No leído"
```

```
    else:
```

```
        print("Opción no válida.")
```

```
    return
```

```
    cursor.execute("UPDATE libros SET estado = ? WHERE id = ?", (nuevo_valor, id_libro))
```

```
    else:
```

```
        print("Opción no válida.")
```

```
    return
```

```
    conexion.commit()
```

```
    print("¡Libro actualizado correctamente!")
```

```
def eliminar_libro():
```

```
    print("\n--- ELIMINAR LIBRO ---")
```

```

try:
    id_libro = int(input("Ingrese el ID del libro: "))
except ValueError:
    print("Debe ingresar un número.")
    return

cursor.execute("SELECT * FROM libros WHERE id = ?", (id_libro,))
libro = cursor.fetchone()

if libro is None:
    print("No se encontró el libro.")
    return

confirmar = input("¿Está seguro de eliminar este libro? (s/n): ").lower()

if confirmar == "s":
    cursor.execute("DELETE FROM libros WHERE id = ?", (id_libro,))
    conexion.commit()
    print("¡Libro eliminado correctamente!")
else:
    print("Operación cancelada.")

def buscar_libros():
    print("\n--- BUSCAR LIBROS ---")
    print("1. Buscar por título")
    print("2. Buscar por autor")
    print("3. Buscar por género")

    opción = input("Seleccione una opción: ")
    texto = input("Ingrese el texto a buscar: ")

    if opción == "1":
        cursor.execute("SELECT * FROM libros WHERE titulo LIKE ?", ("% " + texto + "%",))
    elif opción == "2":
        cursor.execute("SELECT * FROM libros WHERE autor LIKE ?", ("% " + texto + "%",))
    elif opción == "3":
        cursor.execute("SELECT * FROM libros WHERE genero LIKE ?", ("% " + texto + "%",))
    else:
        print("Opción no válida.")
        return

    resultados = cursor.fetchall()

    if not resultados:
        print("No se encontraron libros.")
        return

    print("\n--- RESULTADOS ---")
    for libro in resultados:
        print("-----")
        print("ID:", libro[0])
        print("Título:", libro[1])
        print("Autor:", libro[2])
        print("Género:", libro[3])
        print("Estado:", libro[4])

def menu():
    while True:
        print("\n=====")

```

```

print("    MI BIBLIOTECA")
print("=====")
print("1. Agregar nuevo libro")
print("2. Actualizar libro")
print("3. Eliminar libro")
print("4. Ver listado de libros")
print("5. Buscar libros")
print("6. Salir")
print("=====")

opcion = input("Seleccione una opción: ")

if opcion == "1":
    agregar_libro()
elif opcion == "2":
    actualizar_libro()
elif opcion == "3":
    eliminar_libro()
elif opcion == "4":
    mostrar_libros()
elif opcion == "5":
    buscar_libros()
elif opcion == "6":
    print("\nGracias por utilizar Mi Biblioteca.")
    break
else:
    print("Opción no válida. Intente nuevamente.")

menu()
conexion.close()

```

## 8. Explicación del programa

- **Conexión:** `sqlite3.connect("biblioteca.db")` crea o abre la base de datos.
- **Creación de tabla:** `CREATE TABLE IF NOT EXISTS` crea la tabla libros si no existe.
- **Agregar:** `INSERT INTO` guarda un nuevo libro.
- **Mostrar:** `SELECT` obtiene los libros registrados.
- **Actualizar:** `UPDATE` modifica la información de un libro.
- **Eliminar:** `DELETE` elimina un libro mediante su ID.
- **Buscar:** `LIKE` permite realizar búsquedas parciales por título, autor o género.
- **Menú:** Un ciclo `while` mantiene el programa activo hasta seleccionar la opción Salir.
- **Guardar cambios:** `conexion.commit()` confirma los cambios realizados en SQLite.

## 9. Archivo README.md

#  Mi Biblioteca

## Descripción

Mi Biblioteca es una aplicación de línea de comandos desarrollada en Python que permite administrar una colección personal de libros.

La información se almacena utilizando una base de datos SQLite.

### ### Funcionalidades

- Agregar nuevos libros.
- Actualizar información de libros.
- Eliminar libros.
- Ver todos los libros registrados.
- Buscar libros por título.
- Buscar libros por autor.
- Buscar libros por género.
- Marcar los libros como leídos o no leídos.

### ### Tecnologías utilizadas

- Python
- SQLite
- Biblioteca sqlite3

### ### Requisitos

Tener instalado Python 3.

No es necesario instalar SQLite por separado, ya que Python incluye la biblioteca sqlite3.

### ### Ejecución

1. Abrir una terminal en la carpeta del proyecto.
2. Ejecutar:

```
python biblioteca.py
```

3. Aparecerá el menú principal.

### ### Base de datos

Al ejecutar el programa por primera vez se crea automáticamente el archivo biblioteca.db.

### ### Autor

Proyecto desarrollado como práctica de programación en Python y bases de datos SQLite.

## 10. Resumen del funcionamiento

El usuario inicia el programa y encuentra un menú con seis opciones. Puede registrar libros, consultar el listado, actualizar información, eliminar registros o realizar búsquedas. Todos los cambios se almacenan en la base de datos SQLite biblioteca.db. La opción Salir termina el programa y cierra la conexión con la base de datos.